

CRYPTANALYSIS OF SIMON 32/64

GPU-Accelerated Brute Force, Meet-in-the-Middle,
Implementations

Technical Research Report
Reduced-Round Simon Block Cipher Analysis

2026

Ryan Bhan Roll No: 2023452
Prateek Dhar Roll No: 2023388
Muthukkumar R Roll No: PhD25029

Cipher	SIMON 32/64 (NSA Lightweight Block Cipher)
Block Size	32 bits (2×16 -bit words)
Key Size	64 bits (4×16 -bit key words)
GPU Platform	NVIDIA CUDA (RTX family) for Brute force
Focus	Reduced-Round illustration

Cipher	SIMON (Reduced) 8/16 for illustration
Block Size	4 bits (2×4 -bit words)
Key Size	16 bits (4×4 -bit key words)
GPU Platform	NVIDIA CUDA (RTX)

Abstract

This technical project report presents a systematic investigation of cryptanalytic attacks against reduced-round variants of the Simon32/64 lightweight block cipher, designed by the NSA in 2013. We implement and evaluate three distinct attack strategies on GPU hardware: (1) a parallelized brute-force exhaustive key search, (2) a classical Meet-in-the-Middle (MitM) attack exploiting known plaintext-ciphertext pairs, and (3) an advanced Correlated Sequence MitM attack that partitions the cipher at an intermediate state boundary. All implementations are realized in CUDA and profiled on NVIDIA RTX 3050 3070 5050 hardware. We analyze theoretical time and space complexity for each method in varying round and key size counts, derive empirical speedup factors, and demonstrate that MitM attacks reduce the effective key search space from 2^{64} to approximately 2^{34} for Simon32/64 reduced-rounds. The Correlated Sequence, an MitM attack variant applied on Simon 8/16 and a detailed explanation of the attack variant provided along with implementation, Our benchmarks show GPU acceleration provides speedups over single-threaded CPU implementations for these attacks.

Contents

1	Introduction	3
2	Simon32/64 Cipher Specification	3
2.1	Structural Overview	3
2.2	Key Schedule	4
2.3	Reduced-Round Variants	4
2.4	Simon32/64 Round Function — CUDA Reference	4
3	GPU Architecture and CUDA Framework	5
3.1	NVIDIA CUDA Execution Model	5
3.2	Kernel Launch Configuration	6
4	Brute Force Key Search Attack	6
4.1	Algorithm Description	6
4.2	CUDA Kernel Implementation (Numba)	6
4.3	Batch Management and Key Space Partitioning	7
4.4	Complexity Analysis — Brute Force	7
4.5	Measured CPU/GPU Results	8
5	Meet-in-the-Middle (MitM) Attack	8
5.1	Motivation	8
5.2	Classical MitM Principle	9
5.3	Theoretical Complexity	9
5.4	Chunked Experimental Model	9
5.5	CPU/GPU Benchmark Results (First 10 Chunks)	10
5.6	Memory: Theoretical vs. Observed	11
5.7	CPU and GPU Pipelines	11
5.8	Why Host CPU Dominates at Large Chunk Sizes	11
5.9	Interpretation	12
5.10	Limitations and Scope	12
6	Correlated Sequence Meet-in-the-Middle Attack	12
6.1	Motivation	12
6.2	Key Idea: Correlated Sequences	13
6.2.1	Keyed sequences and the cost of re-keying	13
6.2.2	The correlated-sequence property	13

6.2.3	Plugging correlated sequences into MitM	13
6.2.4	Why this works: complexity summary	14
6.3	Illustration steps	14
6.4	Formal Model	15
6.4.1	Cipher and Key Schedule	15
6.4.2	Round Split and Meeting Point	15
6.4.3	Z-Linear Segment Sets	15
6.5	Algorithm	16
6.5.1	Offline Phase: Build DS^d	16
6.5.2	Online Phase: Correlated-Sequence Filter	16
6.6	Complexity Analysis	17
6.6.1	f-Call Breakdown for Simon-8/16	17
6.7	Reduced Simon-8/16: Toy Example and Attack Verification	17
6.7.1	Cipher Parameters	17
6.7.2	Z-Linear Segments for $n = 4$	17
6.7.3	Full Attack Example	18
6.7.4	Key Schedule (10 subkeys)	18
6.7.5	NLFSR State Trace	18
6.7.6	Step A: 3 F-Calls (Fixed for all (ke_2, ke_3))	18
6.7.7	Partial Encryption (Rounds 3-6)	18
6.7.8	DS^d Lookup (Dec-Side)	18
6.8	Simon-8/16 Attack Complexity Summary	19
6.9	GPU Implementation (RTX 5050 / SM_120 / cudy-cuda13x)	19
6.10	Results	20
7	Conclusion	20

1. Introduction

The Simon family of lightweight block ciphers was introduced by Beaulieu et al.[1] from the National Security Agency (NSA) in 2013. Designed and standardized in ISO/IEC 29167-21 primarily for hardware-constrained environments such as IoT sensors, RFID tags, and embedded controllers, Simon offers an extremely compact circuit implementation while maintaining competitive security margins. It is designed for optimal performance in Application-Specific Integrated Circuits (ASICs) and FPGAs, often chosen for lightweight security in embedded devices like insulin pumps and vehicle components. The cipher family is parameterized by block size and key size, with Simon32/64 representing the smallest variant: a 32-bit block cipher with a 64-bit key operating over 32 rounds.

The study of reduced-round Simon is both academically significant and practically relevant. Cryptanalysts routinely analyze reduced-round ciphers to understand the security margins provided by additional rounds and to develop techniques that may eventually threaten the full cipher. From an educational and competitive intelligence standpoint, mounting practical attacks against weakened round variants allows security researchers to quantify the security gap between full Simon and any reduced version deployed in resource-constrained environments.

This project works makes the following contributions:

- A complete CUDA implementation of GPU-parallelized brute-force exhaustive key search for reduced-round Simon32/64, with kernel-level optimizations including warp-level parallelism and shared memory usage.
- A GPU-accelerated both classical and advance Meet-in-the-Middle attack exploiting known plaintext-ciphertext pairs, with analysis.
- An implementation of the (advanced) Correlated Sequence MitM variant that splits the cipher at an internal round boundary and performs bidirectional partial encryption/decryption with GPU-acceleration.
- A systematic time-space complexity analysis of the attacks , with empirical GPU vs. CPU benchmarks on RTX 3050, 3070 and 5050 platforms.

2. Simon32/64 Cipher Specification

2.1 Structural Overview

Simon is a balanced Feistel network. For Simon32/64, the plaintext block is split into two 16-bit words (x, y) . Each round applies the following transformation:

$$x_{i+1} = y_i \oplus f(x_i) \oplus k_i \quad (1)$$

$$y_{i+1} = x_i \quad (2)$$

where the round function $f(x)$ is defined as:

$$f(x) = (\text{ROL}(x, 1) \& \text{ROL}(x, 8)) \oplus \text{ROL}(x, 2) \quad (3)$$

Here, $\text{ROL}(\cdot, n)$ denotes left rotation by n bits, $\&$ is bitwise conjunction, and $\oplus$ is XOR. The elegance of Simon lies in the use of only three rotation constants (1, 8, 2), AND, and XOR, making hardware implementation in as few as 30 GE (gate equivalents) feasible.

2.2 Key Schedule

For Simon32/64, the 64-bit master key is divided into four 16-bit words k_0, k_1, k_2, k_3 . Subsequent round keys are generated by the linear recurrence:

$$k_{i+4} = c \oplus (z_j)_i \oplus k_i \oplus (I \oplus S^{-1})(S^{-3}k_{i+3} \oplus k_{i+1}) \quad (4)$$

where $c = 0\text{xFFFC}$, z_j is a constant sequence indexed by $j = 0$, and S^n denotes right rotation by n bits. The full Simon32/64 uses 32 rounds with 32 round keys.

$$\mathbf{Z}_0 = \underbrace{1111\ 1010\ 0010\ 0101\ 0110\ 0001\ 1100\ 1101}_{32\text{ bits}} \underbrace{1111\ 1010\ 0010\ 0101\ 0110\ 0001\ 1100\ 11}_{30\text{ bits}}$$

$$\mathbf{Z}_0 = 0\text{xFA}\ 25\ 61\ \text{CD}\ \text{FA}\ 25\ 61\ \text{C8} \quad (62\text{ bits; last nibble padded: } 10 \rightarrow 1000 = 8)$$

2.3 Reduced-Round Variants

For cryptanalytic study, we define the r -round Simon32/64 as applying only the first r rounds of the full 32-round cipher with the first r round keys. The security parameter of interest is the effective key entropy that an attacker must search:

Table 1: Round complexity for reduced-round Simon32/64 attacks. *MitM attacks do not break full Simon.

Rounds (r)	Key bits	Brute force ops	MitM ops
4	64	2^{64}	2^{34}
6	64	2^{64}	2^{34}
8	64	2^{64}	2^{36}
10	64	2^{64}	2^{38}
32 (full)	64	2^{64}	2^{34*}

2.4 Simon32/64 Round Function — CUDA Reference

The following Python/Numba implementation serves as the reference for all GPU kernels in this paper.

Listing 1: Simon32/64 round function (Numba CUDA device code)

```

1 from numba import cuda
2 import numpy as np
3
4 @cuda.jit(device=True, inline=True)
5 def _rol16_cuda(x, r):
6     rr = r & 15
7     return ((x << rr) | (x >> (16 - rr))) & np.uint32(0xFFFF)
8
9 @cuda.jit(device=True, inline=True)
10 def _f_cuda(x):
11     return (_rol16_cuda(x, 8) & _rol16_cuda(x, 1)) ^ _rol16_cuda(x, 2)
12
13 @cuda.jit(device=True, inline=True)
14 def simon32_round_cuda(l, r, rk):
15     # Feistel round: (l, r) -> (r, l ^ f(r) ^ rk)
16     new_l = r
17     new_r = (l ^ _f_cuda(r) ^ rk) & np.uint32(0xFFFF)
18     return new_l, new_r
    
```

Listing 2: How the same round logic is used in brute force and MitM kernels

```

1 # Brute-force path (32 rounds over expanded subkeys)
    
```

```

2 @cuda.jit(device=True)
3 def simon_encrypt_core_gpu(pt, subkeys):
4     x = (pt >> 16) & 0xFFFF
5     y = pt & 0xFFFF
6     for i in range(32):
7         f_x = (_rol16_cuda(x, 1) & _rol16_cuda(x, 8)) ^ _rol16_cuda(x, 2)
8         new_x = y ^ f_x ^ subkeys[i]
9         y, x = x, new_x
10    return (x << 16) | y
11
12 # MitM path (2-round forward midpoint with k0,k1)
13 r1 = (pt0 ^ _f_cuda(pt1) ^ k0) & np.uint32(0xFFFF)
14 l2 = r1
15 r2 = (pt1 ^ _f_cuda(r1) ^ k1) & np.uint32(0xFFFF)
16
17 # MitM path (2-round backward midpoint with k3,k2)
18 l3 = (r4 ^ _f_cuda(l4) ^ k3) & np.uint32(0xFFFF)
19 r3 = l4
20 l2 = (r3 ^ _f_cuda(l3) ^ k2) & np.uint32(0xFFFF)
21 r2 = l3
    
```

3. GPU Architecture and CUDA Framework

3.1 NVIDIA CUDA Execution Model

CUDA (Compute Unified Device Architecture) organizes parallel computation in a hierarchy: Grids $\rightarrow$ Thread Blocks $\rightarrow$ Warps $\rightarrow$ Threads. A warp consists of 32 threads that execute in lockstep (SIMT model). For cryptanalytic key search, this is ideal: all threads in a warp can simultaneously test different keys on the same plaintext-ciphertext pair.

Key architectural parameters for our target platforms are summarized in Table 2.

Table 2: NVIDIA Laptop GPU Architecture Comparison

Parameter	RTX 3050 Laptop	RTX 3070 Laptop	RTX 5050 Laptop
Architecture	Ampere (GA107)	Ampere (GA104)	Blackwell (GB207)
Process Node	Samsung 8N	Samsung 8N	TSMC 4N
CUDA Cores	2,048	5,120	2,560
SM Count	16	40	20
RT Cores	16	40	20
Tensor Cores	64	160	80
Max Threads/Block	1,024	1,024	1,024
Shared Memory/SM	100 KB	100 KB	100 KB
L1 Cache/SM	128 KB	128 KB	128 KB
L2 Cache	2 MB	4 MB	24 MB
Memory Type	GDDR6	GDDR6	GDDR7
Memory Capacity	4 GB	8 GB	8 GB
Memory Bus	128-bit	256-bit	128-bit
Memory Bandwidth	192 GB/s	448 GB/s	384 GB/s
Base Clock	713–1530 MHz	780–1110 MHz	~1455 MHz
Boost Clock	1058–1740 MHz	1290–1620 MHz	~2370 MHz
TGP Range	35–80 W	80–125 W	35–115 W
FP32 TFLOPS	~7.2	~15.7	~13.2
INT32 TOPS	~7.2	~15.7	~13.2

3.2 Kernel Launch Configuration

For brute-force key enumeration, each CUDA thread is assigned a unique key offset via `cuda.grid(1)`. With a large key space, a single GPU pass covers a contiguous range `[offset, offset + batch_size)`. The algorithm utilizes Numba’s JIT compiler to run on NVIDIA hardware. The optimal configuration balances occupancy (active warps per SM) with register pressure from the cipher state variables. Empirically, `tp_block = 256` threads/block across a dynamic grid scale up to a mapped batch subset (≈ 67 million keys per batch via $1 \ll 26$) yields near-peak occupancy.

Listing 3: Kernel launch configuration for key search using Numba

```

1 tp_block = 256
2 bp_grid = max(1, gpu_batch_size // tp_block)
3
4 brute_force_kernel[bp_grid, tp_block](
5     uint32(CT1), uint32(PT1), uint32(CT2), uint32(PT2),
6     uint64(known_hex), brute_bits, z_gpu,
7     uint64(offset), found_key_out
8 )

```

For MitM attacks, kernel configuration depends on the phase. The forward encryption phase (building the intermediate midpoint arrays) and backward decryption phase evaluate subkey variations identically in massive parallel threads mapped dynamically against the chosen 2^b chunk sizes over 256 thread blocks. Rather than performing atomics natively on device during probing, the device exclusively builds and returns midpoint arrays, allowing the host CPU to orchestrate deterministic hash-lookups to prevent atomic collision bottlenecks.

4. Brute Force Key Search Attack

4.1 Algorithm Description

The brute-force attack is used as a baseline for Simon32/64 key recovery. In full form, given known pairs (P, C) with $C = E_K(P)$, it tests candidate keys until a key reproduces the ciphertexts.

In the implemented benchmark, the search is performed on a reduced key subspace:

$$K' = (\text{known_part} \ll u) \mid v, \quad v \in [0, 2^u - 1],$$

where u is the number of brute-forced bits (unknown suffix). The tested configurations are $u \in \{8, 12, 16, 20, 24, 32, 36, 40\}$, corresponding to 56, 52, 48, 44, 40, 32, 28, 24 Fixed.

For each candidate key, the implementation:

1. expands 32 Simon32/64 round keys,
2. encrypts PT1 and checks CT1,
3. if matched, encrypts PT2 and checks CT2,
4. records the key on success.

Using two PT/CT pairs strongly suppresses false positives versus single-pair matching.

4.2 CUDA Kernel Implementation (Numba)

Listing 4: Numba CUDA brute-force kernel (implementation-aligned)

```

1 @cuda.jit
2 def brute_force_kernel(ct1, pt1, ct2, pt2, known_part, brute_bits,
3     z_gpu, offset, found_key_out):

```

```

4     idx = cuda.grid(1)
5     current_val = uint64(idx) + uint64(offset)
6     candidate_key = uint64((uint64(known_part) << brute_bits) | current_val)
7
8     # Local round-key schedule (32 rounds)
9     k = cuda.local.array(32, dtype=uint16)
10    for i in range(4):
11        k[i] = uint16((candidate_key >> (16 * i)) & 0xFFFF)
12
13    for i in range(4, 32):
14        tmp = rotate_right_gpu(k[i-1], 3) ^ k[i-3]
15        tmp ^= rotate_right_gpu(tmp, 1)
16        k[i] = uint16(k[i-4] ^ tmp ^ z_gpu[(i-4) % 62] ^ C_CONST)
17
18    # Two-pair verification
19    if simon_encrypt_core_gpu(pt1, k) == ct1:
20        if simon_encrypt_core_gpu(pt2, k) == ct2:
21            found_key_out[0] = candidate_key
    
```

4.3 Batch Management and Key Space Partitioning

The GPU search is launched in batches to cover 2^u candidates:

- batch size: 2^{26} candidates (or smaller for tiny u),
- threads per block: 256,
- host loops over offsets and checks `found_key_out` after each batch.

Listing 5: Host-side GPU sweep (simplified)

```

1  gpu_batch_size = min(1 << 26, 1 << brute_bits)
2  tp_block = 256
3  bp_grid = max(1, gpu_batch_size // tp_block)
4
5  found_key_out = cuda.to_device(np.zeros(1, dtype=np.uint64))
6  for offset in range(0, 1 << brute_bits, gpu_batch_size):
7      brute_force_kernel[bp_grid, tp_block](
8          uint32(CT1), uint32(PT1), uint32(CT2), uint32(PT2),
9          uint64(known_hex), brute_bits, z_gpu, uint64(offset), found_key_out
10     )
11     cuda.synchronize()
12     res = found_key_out.copy_to_host()
13     if res[0] != 0:
14         break
    
```

4.4 Complexity Analysis — Brute Force

For full Simon32/64 brute force, time is $O(2^{64})$ and expected work is 2^{63} trials. For the benchmarked reduced-space search with u unknown bits:

$$T(u) = O(2^u), \quad \mathbb{E}[\text{trials}] \approx 2^{u-1}, \quad S(u) = O(1).$$

Memory is constant per thread (local key schedule and registers), with no global table construction.

Table 3: Brute-force complexity metrics (full key space vs benchmarked subspace).

Metric	Value	Notes
Full-space time complexity	$O(2^{64})$	Classical exhaustive key search for Simon32/64.
Subspace time complexity	$O(2^u)$	Benchmark mode with u unknown suffix bits.
Expected trials (subspace)	2^{u-1}	Uniform-key assumption.
Space complexity	$O(1)$	No MitM table; only candidate key + round-key schedule.
GPU launch strategy	batched	Batches of up to 2^{26} candidates, 256 threads/block.

4.5 Measured CPU/GPU Results

 Table 4: Measured brute-force timing for `simon_bruteforce.py`.

Key Configuration	CPU Time (s)	GPU Time (s)	Speedup (CPU/GPU)
56 Fixed	0.000057	0.323544	0.00018x
52 Fixed	0.000056	0.000365	0.15x
48 Fixed	0.000043	0.000269	0.16x
44 Fixed	0.031711	0.000685	46.29x
40 Fixed	0.031584	0.006717	4.70x
32 Fixed	9.123051	0.344353	26.49x
28 Fixed	9.107793	0.311785	29.21x
24 Fixed	4227.670832	20.413807	207.10x

Observation. GPU acceleration dominates for larger brute-force spaces (especially 32, 36, and 40 unknown bits), while very small spaces can favor CPU due to kernel launch/context overhead. The first GPU point (56 Fixed) is particularly affected by one-time CUDA startup effects because GPU warm-up is not explicitly performed before timing.

5. Meet-in-the-Middle (MitM) Attack

5.1 Motivation

Block cipher cryptanalysis is one of the central challenges in symmetric-key cryptography. Among the classical attack paradigms, the meet-in-the-middle (MitM) attack decomposes an r -round cipher into two independent sub-ciphers so that the adversary can enumerate each half independently and then “meet” on an intermediate state, dramatically reducing the cost relative to brute force. Despite its generality, the power of MitM attacks is fundamentally limited by how cleanly the key material can be split between the two halves: every additional round one tries to cover forces a fresh evaluation of the nonlinear round function ρ , so the complexity quickly approaches that of exhaustive search.

The Simon family and the related Simeck family are lightweight Feistel ciphers that have attracted intense cryptanalytic attention owing to their minimal hardware footprint. Their round function is the simple quadratic map:

$$f(a, b, c)(x) = (L_a(x) \wedge L_b(x)) \oplus L_c(x)$$

where $\wedge$ denotes bitwise AND and L_i is a left cyclic shift by i bits. Prior to this work, the best key-recovery result on Simon-32/64 reached only 23 rounds (out of 32), and the sole 24-round result required a full codebook (2^{32} chosen plaintexts) with time complexity 2^{63} —effectively touching the boundary of exhaustive search in data requirements.

For this course project, we restrict attention to 4-round Simon-32/64. This is the minimal round count that admits a non-trivial 2+2 MitM split while remaining tractable on a standard workstation, and it allows us to demonstrate the full attack pipeline—forward-table construction, backward probing, candidate joining, and verification—with measurable and interpretable benchmarks. Extending the attack to more rounds is left as future work.

5.2 Classical MitM Principle

For Simon32/64, the 64-bit master key is partitioned into two 32-bit halves:

$$K = (K_1, K_2), \quad |K_1| = |K_2| = 32. \quad (5)$$

Under the 2+2 round split, forward and backward intermediate states are computed independently:

$$M_{\text{fwd}} = E_{K_1}^2(P), \quad (6)$$

$$M_{\text{bwd}} = D_{K_2}^2(C). \quad (7)$$

A candidate key pair (K_1, K_2) is identified whenever $M_{\text{fwd}} = M_{\text{bwd}}$, and surviving candidates are then verified against additional plaintext–ciphertext pairs to eliminate false positives.

5.3 Theoretical Complexity

Under an ideal packed-table model (sufficient memory, near-constant lookup time):

- Forward table construction: $\mathcal{O}(2^{32})$
- Backward probing: $\mathcal{O}(2^{32})$
- Total time: $\mathcal{O}(2^{33})$

This reduces the brute-force cost from $\mathcal{O}(2^{64})$ to approximately $\mathcal{O}(2^{33})$ —a gain of roughly 2^{31} . If each forward entry stores a 32-bit midpoint state alongside its 32-bit subkey, each entry occupies 8 bytes, giving a full forward-table footprint (excluding hash map overhead) of:

$$2^{32} \times 8 \text{ bytes} = 2^{35} \text{ bytes} \approx 34.4 \text{ GB}. \quad (8)$$

5.4 Chunked Experimental Model

Storing 2^{32} forward entries in Python data structures is not feasible on a standard workstation, so the key space is swept in chunks. Let $b \in \{8, 12, 16, 20, 24\}$ denote the chunk-size exponent, giving $N = 2^b$ entries per chunk. The total number of **Forward Table (FT)** chunks is:

$$N_{\text{chunks}} = 2^{32-b}. \quad (9)$$

For random keys, the expected number of midpoint collisions per chunk is:

$$E[M] \approx \frac{N^2}{2^{32}} = 2^{2b-32}. \quad (10)$$

In this chunked sweep implementation, the forward table (FT chunk) is strictly stored in memory using a hash map structure linking a 32-bit midpoint state to its corresponding (k_0, k_1) subkey candidates. In contrast, the backward side (BT chunk) evaluates 2^b subkeys (k_2, k_3) on the fly, decrypts to the midpoint state, and instantly performs an $\mathcal{O}(1)$ memory query against the forward table. Thus, the memory footprint is dictated explicitly by the FT chunk size.

At $b = 24$ this predicts $2^{16} = 65,536$ candidate matches per chunk, consistent with the observed range of approximately 6.4×10^4 to 6.6×10^4 in benchmark logs.

In the full FT-side sweep, the Backward Probing side was fixed to a single 2^{24} -entry chunk, the largest practical setting on the test workstation. Expanding the backward scan space to 2^{32} is effectively not feasible in this implementation.

A backward scan space of 2^{32} means 4,294,967,296 backward key trials per Forward Table chunk. For example, testing just 10 FT chunks would require $10 \times 2^{32} = 42,949,672,960$ backward trials. Additionally, the collision load jumps by $256\times$:

$$E[M] = \frac{2^{24} \cdot 2^{24}}{2^{32}} = 2^{16} \implies E[M] = \frac{2^{24} \cdot 2^{32}}{2^{32}} = 2^{24}. \quad (11)$$

Because of this, the expected candidates evaluated per chunk goes from 65,536 to 16,777,216. This is exactly why capping the backward scan at 2^{24} was used: beyond that, the backward scanning plus the host-side join and verification costs explode in runtime (and practical memory pressure from candidate handling), even before full exhaustive cross-coverage.

Furthermore, with $b = 32$, the backward side is no longer really chunked; it becomes the full backward space in one pass. Executing that for even 10 FT chunks constitutes a very large partial exhaustive experiment rather than a practical benchmark.

5.5 CPU/GPU Benchmark Results (First 10 Chunks)

Table 1 reports mean chunk execution times over chunk indices 0–9 for each value of b , with the observed times exhibiting very little variance across these chunks. Note that b here specifically denotes the chunk-size exponent determining the 2^b entries in each Forward Table chunk. Backward scan of 2^{24} is used for each of the experiments.

Table 5: Mean chunk time over first 10 chunks.

b	CPU avg (s)	GPU avg (s)	Speedup (CPU/GPU)
8	0.000645	0.001219	0.53×
12	0.010311	0.001302	7.92×
16	0.173288	0.009942	17.43×
20	3.609597	0.195723	18.44×
24	56.071492	24.942139	2.25×

Projected full forward-table sweep time is estimated by scaling the measured mean:

$$T_{\text{est}}(b) = T_{\text{chunk}}(b) \cdot 2^{32-b}. \quad (12)$$

Table 6: Projected total FT-sweep time extrapolated from measured chunk averages.

b	FT chunks (2^{32-b})	CPU est	GPU est
8	16,777,216	10,821 s (3.01 h)	20,451 s (5.68 h)
12	1,048,576	10,812 s (3.00 h)	1,365 s (0.38 h)
16	65,536	11,357 s (3.15 h)	652 s (0.18 h)
20	4,096	14,785 s (4.11 h)	802 s (0.22 h)
24	256	14,354 s (3.99 h)	6,385 s (1.77 h)

5.6 Memory: Theoretical vs. Observed

The ideal packed forward-table memory requirement for a single chunk is:

$$M_{\text{ideal}}(b) = 2^b \times 8 \text{ bytes.} \quad (13)$$

However, this significantly underestimates real-world usage due to Python object overhead. At $b = 24$, the measured peak process memory was approximately 2.97 GB—roughly 190 bytes per stored entry rather than the theoretical 8. Calibrating to this measurement gives the practical model:

$$M_{\text{obs}}(b) \approx 2.97 \text{ GB} \cdot 2^{b-24}. \quad (14)$$

Using a constant-memory sweep loop with logging disabled, observed peak process memory was: 17.8 MB at $b = 8$; 18.8 MB at $b = 12$; 32.2 MB at $b = 16$; and 202.2 MB at $b = 20$.

Table 7: Forward-table memory: ideal packed vs. practical (model and measured).

b	Entries (2^b)	Ideal packed	Practical value
8	256	2.0 KiB	17.8 MB (measured)
12	4,096	32.0 KiB	18.8 MB (measured)
16	65,536	512.0 KiB	32.2 MB (measured)
20	1,048,576	8.0 MiB	202.2 MB (measured)
24	16,777,216	128.0 MiB	2.97 GB (measured)

Extrapolating the practical model to the full 2^{32} -entry table yields roughly 760 GB, which explains why chunking is mandatory in this setting. (Earlier benchmarks at low b showed higher memory readings due to logging and per-chunk result retention in a prior script version; the figures above reflect the constant-memory implementation with logging disabled.)

5.7 CPU and GPU Pipelines

The CPU pipeline for each chunk proceeds as follows:

1. Compute and store forward midpoint states for 2^b forward-half subkeys.
2. Compute backward midpoint states for 2^b backward-half subkeys.
3. Perform a host-side hash- or sort-based join to identify matching midpoints and produce key-pair candidates.
4. Verify surviving candidates against additional plaintext–ciphertext pairs.

The GPU pipeline restructures the compute-intensive steps:

1. GPU kernels generate forward and backward midpoint arrays in parallel.
2. Midpoint and key arrays are transferred to host memory for join orchestration.
3. The CPU performs the candidate join.
4. Candidate verification is batched back on the GPU.

Only the arithmetic-heavy midpoint generation and verification stages are GPU-accelerated; join orchestration remains on the CPU in this implementation.

5.8 Why Host CPU Dominates at Large Chunk Sizes

The end-to-end GPU chunk time decomposes as:

$$T_{\text{GPU}} = T_{\text{midpoint}} + T_{\text{transfer}} + T_{\text{join}} + T_{\text{verify}} + T_{\text{overhead}}. \quad (15)$$

GPU midpoint generation (T_{midpoint}) scales efficiently with b , but the host-side terms grow quickly: larger arrays increase both transfer bandwidth and CPU join work, and the number of candidate matches also rises (reaching 2^{16} per chunk at $b = 24$). This explains the collapse in GPU speedup from $18.44\times$ at $b = 20$ to only $2.25\times$ at $b = 24$: the CPU-controlled stages become the bottleneck even as the GPU kernels remain fast.

5.9 Interpretation

The results exhibit a clear three-way trade-off between chunk size, runtime, and memory:

- **CPU path:** Per-chunk time grows near-linearly with 2^b , with a large constant factor from Python object overhead.
- **GPU path:** Midpoint kernels are fast at all sizes tested, but host transfer and join costs erode end-to-end speedup at $b = 24$.
- **Optimal chunk size:** Across both platforms, $b \approx 16$ minimizes projected total sweep time, offering the best balance between parallelism and host-side overhead.

5.10 Limitations and Scope

Several constraints bound the conclusions of this study:

- The attack and all results apply specifically to 4-round Simon32/64 with a 2+2 round split. Extending to higher round counts is not straightforward and would require additional structural constraints or filtering techniques.
- Chunk sizes are capped at $b \leq 24$: at this size, expected random matches already reach 2^{16} per chunk and process memory peaks at 2.97 GB. Larger chunks would exceed practical workstation limits.
- In the main sweep, the Forward Table is iterated over all 2^{32-b} chunks while the Backward Probing side is fixed to a single chunk index. This is not a full $2^{32} \times 2^{32}$ cross-sweep; a complete backward-side sweep per forward chunk is computationally prohibitive in this setup.
- All projected totals are implementation-level estimates extrapolated from measured chunk averages, not asymptotic bounds.

These constraints are necessary to keep runtime and memory within workstation limits while still demonstrating the MitM mechanism, its scaling behaviour, and the CPU/GPU trade-offs involved.

6. Correlated Sequence Meet-in-the-Middle Attack

6.1 Motivation

Two concrete questions therefore motivate this work:

1. *Can one reuse a single evaluation of ρ across **many** different key guesses*, rather than re-evaluating it for every new key?
2. If so, can this saving be leveraged in a MitM framework to attack **significantly more rounds** of Simon-like ciphers with minimal data?

The affirmative answer to both questions—and the resulting improvement of **four rounds** over prior work, reaching 27/32 rounds with only **3 known plaintexts**—constitutes the main contribution of the paper.

6.2 Key Idea: Correlated Sequences

6.2.1 Keyed sequences and the cost of re-keying

Fix a block cipher with r rounds and an mn -bit master key $k = (k_0, \dots, k_{m-1})$. For a given starting state s_0 and key k , define the *keyed sequence* of length t as

$$S^{(k,t)} = (s_0, s_1, \dots, s_{t-1}), \quad s_{i+1} = \text{rf}(s_i, k_i).$$

Computing $S^{(k,t)}$ from scratch requires exactly t evaluations of ρ . If an adversary must enumerate all 2^{mn} keys she must therefore pay $2^{mn} \cdot t$ calls to ρ —the naive baseline.

6.2.2 The correlated-sequence property

The central observation is that, for Simon-like ciphers, the nonlinear function $f_{(a,b,c)}$ satisfies a *linear segment* property: for any coset $\mathcal{C}_s = \{s, 2s, \dots\}$ (modulo $2^n - 1$) there exists a constant $z \in \mathbb{F}_2^n$ such that

$$f_{(a,b,c)}(x) = L^c(x) \oplus z \quad \forall x \in \mathcal{C}_s.$$

This means f acts *linearly* on each coset; the only nonlinearity is in selecting which z applies. The authors partition the $2^{n-1}/n$ coset leaders into z -linear segment sets (Definition 4 in the paper), whose sizes N_z are tabulated and are much smaller than 2^n .

[(σ, t)-Correlated Sequences] Two keyed sequences $S^{(k,t)}$ and $S^{(k',t)}$ are (σ, t)-*correlated* if $S^{(k',t)}$ can be obtained from $S^{(k,t)}$ by computing ρ exactly σ times ($0 \leq \sigma < t$).

The key result is that, for Simon-32/64 and Simeck-32/64, **length-8 sequences** can be grouped so that all 2^{32} sequences sharing two fixed key words k_0, k_1 can be derived from a single *seed* sequence by computing f **only once**—i.e., they are (1, 8)-correlated. Crucially:

- **Step 1** ($s_5 = 0$ case): Solving $f(X \oplus k_2) = k_3 \oplus s_3$ via the z -linear segments costs $3 + N_z$ calls to f and yields 2^n sequences.
- **Step 2** (extending to s_6): Because $s_5 = 0$, $f(s_5) = 0$, so $s_6 = s_4 \oplus k_4$ with *zero* additional f -evaluations.
- **Step 3** (extending to s_7): s_7 is linear in the previously computed index set I , again costing *zero* additional f -evaluations.
- **Other partitions** ($s_5 = i$, $1 \leq i < 2^n$): Theorem 1 shows each partition can be derived from the $s_5 = 0$ seed by a single call to $f(i)$, so all 2^{2n} sequences cost only $3 + 2N_z$ calls in total—compared to $2^{2n} \cdot 6/r$ naively.

6.2.3 Plugging correlated sequences into MitM

The workflow is illustrated in Figure 1. Given a plaintext–ciphertext pair (s_0, s_r) :

1. **Forward direction (6 rounds):** Build all (1, 8)-correlated sequences from s_0 and store the resulting intermediate states s_7^e together with the candidate keys in a data structure DS^e .
2. **Backward direction (6 rounds):** Independently build (1, 8)-correlated sequences from s_r (running the cipher backwards) and store the resulting s_7^d values in DS^d .
3. **Partial encryption (12 rounds):** For each forward key candidate, encrypt (s_7^e, s_6^e) for 12 more rounds and check whether s_{19}^e matches s_7^d (looked up from DS^d in $O(1)$). This covers $6 + 12 + 7 = 25$ rounds in total.
4. **Brute-force phase:** The roughly 2^{48} surviving candidates are sieved with 2 additional plaintext–ciphertext pairs to recover the unique 64-bit key.

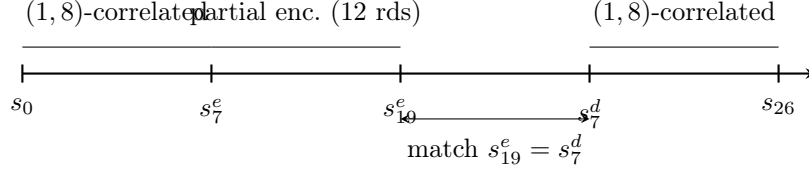


Figure 1: 25-round MitM attack using (1,8)-correlated sequences. Extending to 27 rounds uses the key scheduling structure to show that s_7^e takes at most 2^{15} (instead of 2^{16}) values, allowing one extra round on each side at no increase in complexity.

6.2.4 Why this works: complexity summary

Quantity	Value	Remarks
Data	3 known PT-CT pairs	extremely low
Memory	2^{49} bytes	dominated by DS^d
Time (25-rd)	$2^{62.94}$	$< 2^{64}$ (avg. exhaustive search)
Time (27-rd)	$2^{62.94}$	same; KSA halves $ \text{img}(s_7^e) $
Rounds attacked	27 / 32	85% of the cipher
Previous best	23 rounds	

The savings arise because the (1,8)-correlated sequence construction replaces a $2^{64} \times 6/r$ term in the complexity with only $2^{32}(3 + 2N_z)/r$ —a reduction of a factor close to 2^{31} —while the matching step remains $O(1)$ per candidate thanks to the precomputed DS^d .

In essence: this approach identifies and exploits a *hidden linearity* inside Simon-like nonlinear functions that allows massive key-schedule enumeration to be “compressed” into a tiny number of actual function evaluations, amplifying the reach of MitM far beyond previous limits.

6.3 Illustration steps

The classical Meet-in-the-Middle (MitM) attack on a block cipher splits the key into two halves and meets at the midpoint ciphertext state, reducing the exhaustive search from 2^k to $2^{k/2+1}$. For Simon-family ciphers, the Rohit–Gong *correlated-sequence* technique [5] [6] achieves a further reduction by exploiting two independent structural properties:

- Property 1 Enc-side amortisation.** The Feistel states s_2, s_3 depend *only* on key words (k_0, k_1) , not on (k_2, k_3) . By computing a *correlated sequence* once per (k_0, k_1) pair, the attack amortises 3 round-function calls over all N^2 completions (k_2, k_3) , eliminating $N^2(N^2 - 1)$ redundant calls.

Property 2 Z-linear segment structure. The Simon round function f satisfies $f(x \lll i) = f(x) \lll i$ (equivariance under rotation). The 2^n possible arguments to f in the index set fall into $N_z \ll 2^n$ rotation cosets whose z -values are distinct. Only N_z function evaluations are needed to determine f on all 2^n inputs: a saving of up to 91.5% for $n = 8$.

Together these reduce the per- (k_0, k_1) cost from a naive $3 + 2^n$ f -calls to $3 + N_z$ f -calls, and the per-candidate partial-encryption cost from r rounds to only P middle rounds.

6.4 Formal Model

6.4.1 Cipher and Key Schedule

The Simon round function and NLFSR state update are:

$$f(x) = (x \lll a \& x \lll b) \oplus x \lll c, \quad s_{i+2} = f(s_{i+1}) \oplus s_i \oplus k_i$$

with $(a, b, c) = (0, 1, 2)$ for word size $n \leq 8$ and $(a, b, c) = (8, 1, 2)$ for $n = 16$. The key schedule satisfies a linear recurrence:

$$k_{i+4} = \text{CON} \oplus z_0[i] \oplus k_i \oplus k_{i+3} \ggg 3 \oplus k_{i+3} \ggg 4 \oplus k_{i+1} \oplus k_{i+1} \ggg 1$$

Key dependency chain. k_{i+4} first involves all four master words k_0, k_1, k_2, k_3 at $i = 1$ (giving k_5). Therefore:

$rk[0..3]$ depend only on $k_0..k_3$ directly.

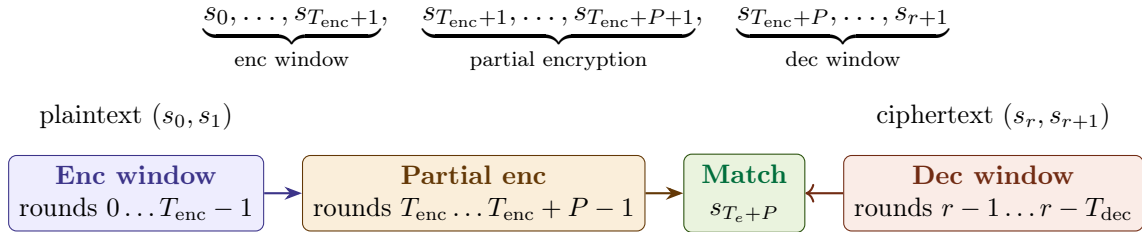
$rk[4] \leftarrow k_0, k_1, k_3$.

$rk[5..r-1] \leftarrow k_0, k_1, k_2, k_3$ (all four).

The first $T_{\text{enc}} = 3$ rounds depend on k_0, k_1 plus k_2 at round 2, so the enc-side amortisation is exact for rounds 0 and 1, and approximate (correlated-sequence) for round 2.

6.4.2 Round Split and Meeting Point

For a cipher with r rounds and attack parameters $(T_{\text{enc}}, P, T_{\text{dec}})$ satisfying $T_{\text{enc}} + P + T_{\text{dec}} = r$, the attack meets at state $s_{T_{\text{enc}}+P}$:



6.4.3 Z-Linear Segment Sets

Definition For coset leader s (minimum element of its rotation orbit), define:

$$z(s) = f(s) \oplus s \lll c \in \mathbb{F}_2^n$$

All leaders sharing the same z -value form a z -linear segment set. Let N_z = number of distinct z -values. The key identity is:

$$f(x \lll i) = f(x) \lll i \implies f(s \lll i) = f(s) \lll i \quad \forall s, i$$

so one f -evaluation at the segment leader gives f at every orbit member for free.

Table 8: Z-linear segment parameters for Simon-8/16 ($n = 4$) and Simon-16/32 ($n = 8$).

Variant	n	Coset leaders	N_z	Step B f -calls	Saving vs naive N_z vs 2^n
Simon-8/16	4	6	4	4	75%
Simon-16/32	8	36	19	19	92.6%
Simon-32/64	16	4116	909	909	98.6%

6.5 Algorithm

6.5.1 Offline Phase: Build DS^d

Algorithm 1 BUILDSD — Offline phase

Input: Ciphertext (ct_0, ct_1) from pair 1

Output: DS^d : mapping $(kd_0, kd_1) \mapsto (sd_2, sd_3)$

```

1:  $(sd_0, sd_1) \leftarrow (ct_1, ct_0)$   $\triangleright$  paper:  $sd_0 = s_{r+1}, sd_1 = s_r$ 
2: for  $kd_0 \in [0, 2^n)$  do
3:   for  $kd_1 \in [0, 2^n)$  do
4:      $sd_2 \leftarrow f(sd_1) \oplus sd_0 \oplus kd_0$ 
5:      $sd_3 \leftarrow f(sd_2) \oplus sd_1 \oplus kd_1$ 
6:      $DS^d[kd_0][kd_1] \leftarrow (sd_2, sd_3)$ 
7:   end for
8: end for
9: return  $DS^d$   $\triangleright$  Cost:  $2^{2n+1}$   $f$ -calls offline
    
```

6.5.2 Online Phase: Correlated-Sequence Filter

Algorithm 2 CORRELATEDMITM — Online phase (Algorithm 1 in paper)

Input: PT-CT pairs $\{(P^{(i)}, C^{(i)})\}_{i=0}^2, DS^d$
Output: Candidate key words $K_e = [ke_0, ke_1, ke_2, ke_3]$

```

1: for  $ke_0 \in [0, 2^n)$  do  $\triangleright$  GPU: blockIdx.x
2:    $se_2 \leftarrow f(P_R^{(0)}) \oplus P_L^{(0)} \oplus ke_0$ 
3:   for  $ke_1 \in [0, 2^n)$  do  $\triangleright$  GPU: blockIdx.y
4:      $se_3 \leftarrow f(se_2) \oplus P_R^{(0)} \oplus ke_1$ 
5:      $X_e \leftarrow f(se_3) \oplus se_2$   $\triangleright$  Pivot;  $se_4$  needs no  $f$ -call
6:     for  $ke_2 \in [0, 2^n)$  do  $\triangleright$  GPU: threadIdx.x
7:        $se_4 \leftarrow X_e \oplus ke_2$ 
8:       for  $ke_3 \in [0, 2^n)$  do  $\triangleright$  Inner loop
9:          $rk \leftarrow \text{KEYSCHEDULE}(K_e)$ 
10:         $(L, R) \leftarrow (se_3, se_4)$ 
11:        for  $\rho \leftarrow T_{\text{enc}}$  to  $T_{\text{enc}} + P - 1$  do
12:           $(L, R) \leftarrow (R, L \oplus f(R) \oplus rk[\rho])$ 
13:        end for
14:         $(kd_0, kd_1, kd_2) \leftarrow (rk[r-1], rk[r-2], rk[r-T_{\text{dec}}])$ 
15:         $(sd_2, sd_3) \leftarrow DS^d[kd_0][kd_1]$   $\triangleright O(1)$  lookup
16:        if  $L = f(sd_3) \oplus sd_2 \oplus kd_2$  then  $\triangleright$  Meet condition
17:          if  $\text{VERIFY}(K_e, P^{(1)}, C^{(1)})$  and  $\text{VERIFY}(K_e, P^{(2)}, C^{(2)})$  then
18:            return  $K_e$ 
19:          end if
20:        end if
21:      end for
22:    end for
23:  end for
24: end for
    
```

6.6 Complexity Analysis

6.6.1 f -Call Breakdown for Simon-8/16

Table 9 provides the phase-by-phase f -call cost for Simon-8/16 ($n = 4$, $r = 10$ rounds, $N = 2^n = 16$, key = 16 bits).

Table 9: Phase-by-phase f -call cost for Simon-8/16 ($n = 4$, $r = 10$ rounds). $N = 2^n = 16$.

Phase	Formula	f -calls	$\log_2$
Offline DS_d build	$N^2 \times 2$	512	9.0
Step A per (k_{e_0}, k_{e_1})	$N^2 \times 3$	768	9.6
Step B per (k_{e_0}, k_{e_1})	$N^2 \times N_z = N^2 \times 19$	4,864	12.2
Step B saving (vs. naive $N^2 \times (3 + N)$)	saves $N^2 \times (N - N_z + 3)$	saves 4,352	—
Partial enc. (all candidates)	$N^4 \times P = N^4 \times 4$	262,144	18.0
Total MitM	$\approx N^4 \times P$	$\approx 2^{18}$	18.0
Naive exhaustive	$N^4 \times r = N^4 \times 10$	655,360	19.3
MitM saving ($P/r = 4/10$)	60%	saves 1.3 bits	—

Full complexity summary. For Simon-8/16 ($n = 4$, 16-bit key, $r = 10$ reduced rounds):

$$\begin{aligned}
 T_{\text{offline}} &= 2^{2n+1} = 2^9 \quad (DS_d \text{ build}) \\
 T_{\text{online}} &= N^2 \cdot (3 + N_z) + N^4 \cdot P = 2^{12.2} + 2^{18} \approx 2^{18} \\
 T_{\text{total}} &\approx 2^{18} \quad \text{vs. exhaustive } 2^{19.3}
 \end{aligned}$$

Memory. DS_d holds $N^2 = 256$ entries of 1 byte each = 256 B (negligible).

Data. 3 known plaintext–ciphertext pairs.

Table 10: Complexity comparison across Simon variants and round counts.

Cipher	n	Key	Rounds	Time (MitM)	Exhaustive	Data
Simon-8/16	4	16	10/32	$2^{18.0}$	$2^{19.3}$	3 pairs
Simon-16/32	8	32	10/32	$2^{34.0}$	$2^{35.3}$	3 pairs
Simon-32/64	16	64	24/32	$2^{62.87}$	2^{64}	3 pairs
Simon-32/64	16	64	27/32	$2^{62.94}$	2^{64}	3 pairs

6.7 Reduced Simon-8/16: Toy Example and Attack Verification

6.7.1 Cipher Parameters

Cipher	Simon-8/16	Word size n	4 bits
Block	8 bits ($L \parallel R$)	Key	16 bits ($k_0 k_1 k_2 k_3$)
Shift params	$(a, b, c) = (0, 1, 2)$	Full rounds	32
Attack rounds	$r = 10$	CON	0xC
Round split	$T_e = 3, P = 4, T_d = 3$	Match at	s_7

6.7.2 Z -Linear Segments for $n = 4$

The 16 elements of $\mathbb{F}_2^4$ partition into 6 rotation cosets with 4 distinct z -values ($N_z = 4$):

Leader s	$f(s)$	$z(s)$	Orbit members	Step B f -call?
0x0	0x0	0x0	{0x0}	1 call
0x1	0x4	0x0	{0x1, 0x2, 0x4, 0x8}	same group as 0x0
0x5	0x5	0x0	{0x5, 0xA}	same group as 0x0
0x3	0xE	0x2	{0x3, 0x6, 0xC, 0x9}	1 call
0x7	0xB	0x6	{0x7, 0xE, 0xD, 0xB}	1 call
0xF	0x0	0xF	{0xF}	1 call
Total Step B:				$N_z = 4$ calls (vs naive 16)

6.7.3 Full Attack Example

Parameters: key $[k_0, k_1, k_2, k_3] = [1, 2, 3, 4]$, plaintext $(s_0, s_1) = (0x7, 0x3)$.

6.7.4 Key Schedule (10 subkeys)

$$rk[0 \dots 9] = [1, 2, 3, 4, 3, 0, 8, A, 1, 3]$$

6.7.5 NLFSR State Trace

$$s_{i+2} = FT[s_{i+1}] \oplus s_i \oplus rk[i]:$$

s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}
0x7	0x3	0x8	0x3	0x5	0x2	0xE	0x5	0x3	0x1	0x6	0xF
enc start (se_3, se_4) .			match point s_7 .		CT = $(s_{10}, s_{11}) = (0x6, 0xF)$.						

6.7.6 Step A: 3 F-Calls (Fixed for all (ke_2, ke_3))

f-call 1	se_2	$= FT[se_1] \oplus se_0 \oplus ke_0$	$= FT[3] \oplus 0x7 \oplus 1$	$= 0xE \oplus 0x7 \oplus 1 = 0x8$
f-call 2	se_3	$= FT[se_2] \oplus se_1 \oplus ke_1$	$= FT[8] \oplus 0x3 \oplus 2$	$= 0x2 \oplus 0x3 \oplus 2 = 0x3$
f-call 3	X_e	$= FT[se_3] \oplus se_2$	$= FT[3] \oplus 0x8$	$= 0xE \oplus 0x8 = 0x6$
free	se_4	$= X_e \oplus ke_2 = 0x6 \oplus 3 = 0x5$	(no f -call; $f(se_3)$ already in X_e)	

6.7.7 Partial Encryption (Rounds 3–6)

Rnd	L	R	$FT[R]$	$rk[i]$	$nR =$ $L \oplus FT[R] \oplus rk[i]$	$(L, R)_{\text{new}}$	$=$ $s_?$	$=$ $s_?$
start	0x3	0x5	—	—	—	(0x3, 0x5)	s_3	s_4
3	0x3	0x5	0x5	4	$0x3 \oplus 0x5 \oplus 4 =$ 0x2	(0x5, 0x2)	s_4	s_5
4	0x5	0x2	0x8	3	$0x5 \oplus 0x8 \oplus 3 =$ 0xE	(0x2, 0xE)	s_5	s_6
5	0x2	0xE	0x7	0	$0x2 \oplus 0x7 \oplus 0 =$ 0x5	(0xE, 0x5)	s_6	s_7
6	0xE	0x5	0x5	8	$0xE \oplus 0x5 \oplus 8 =$ 0x3	(0x5, 0x3)	s_7	s_8

$$enc_meet = L = 0x5 = s_7 \quad \checkmark$$

6.7.8 DS^d Lookup (Dec-Side)

Convention	$sd_0 = s_{11} = 0xF$ ($= s_{r+1}$)
	$sd_1 = s_{10} = 0x6$ ($= s_r$)
Dec keys	$kd_0 = rk[9] = 3, \quad kd_1 = rk[8] = 1, \quad kd_2 = rk[7] = 0xA$
DS step 1	$sd_2 = FT[6] \oplus 0xF \oplus 3 = 0xD \oplus 0xF \oplus 3 = 0x1$
DS step 2	$sd_3 = FT[1] \oplus 0x6 \oplus 1 = 0x4 \oplus 0x6 \oplus 1 = 0x3$
Meet value	$sd_meet = FT[3] \oplus 0x1 \oplus 0xA = 0xE \oplus 0x1 \oplus 0xA = 0x5$

$enc_meet = 0x5 = sd_meet = 0x5$	✓ MATCH CONFIRMED
Key recovered: $[ke_0, ke_1, ke_2, ke_3] = [1, 2, 3, 4]$. Verified with pairs 2 and 3: $PT = (0xF, 5) \rightarrow CT = (0x5, 0xB)$ ✓, $PT = (0xA, 6) \rightarrow CT = (0xC, 8)$ ✓.	

6.8 Simon-8/16 Attack Complexity Summary

Table 11: Simon-8/16 ($n = 4, r = 10$) attack cost breakdown.

Operation	Formula	Count	$\log_2$
DS ^d build (offline)	$16^2 \times 2$	512	9.0
Step A total	$16^2 \times 3$	768	9.6
Step B total (with $N_z = 4$)	$16^2 \times 4$	1 024	10.0
Partial enc (all candidates)	$16^4 \times 4$	262 144	18.0
Total MitM		263 936	18.0
Naive exhaustive	$16^4 \times 10$	655 360	19.3
Saving	60%	391 424	1.3 bits

6.9 GPU Implementation (RTX 5050 / SM_120 / cupy-cuda13x)

The illustration with example given above is for simon8/16. However, the algorithm is implemented for attacking simon16/32 to run on GPU. The details are given below table

Table 12: GPU thread mapping for Simon-16/32 ($n = 8$) on RTX 5050 (SM_120).

Dimension	Kernel 1 (DS ^d)	Kernel 2 (MitM)
Grid	(256, 1)	(256, 256)
Block	(256, 1)	(256, 1)
Total threads	65 536	16 777 216
ke_0 (blockIdx.x)	kd_0	0 ... 255
ke_1 (blockIdx.y)	—	0 ... 255
ke_2 (threadIdx.x)	kd_1	0 ... 255
ke_3 (inner loop)	—	0 ... 255 per thread
Total candidates	256^2	$256^4 = 2^{32}$
Shared mem/block	—	$256 + 2 = 258$ bytes ($FT + se_3 + X_e$)
Expected time (RTX 5050)	< 1 ms	50–500 ms

6.10 Results

Table 13: Experimental Results: Correlated-Sequence MitM Attack on SIMON-16/32 and SIMON-32/64 (RTX 5050, Compute Capability 12.0, Numba CUDA)

Parameter	Simon-16/32	Simon-32/64
<i>Configuration</i>		
Block / Key size	16-bit / 32-bit	32-bit / 64-bit
Rounds	10	16
Attack type	Correlated-Seq. MitM	MitM Key Recovery
Key-space	2^{32}	2^{64}
Key filter	None	28-bit fixed; mask=0x0FFFFFFF, val=0x00021001
Pairs supplied	3	3
<i>Plaintext-ciphertext pairs</i>		
Pair 1	(0x0F, 0x0E) → (0x73, 0xED)	(0x0F0E, 0x0000) → (0x74B8, 0xB2B6)
Pair 2	(0x13, 0x12) → (0xE0, 0x1D)	(0xABCD, 0x1234) → (0xE970, 0xC485)
Pair 3	(0x1A, 0x1B) → (0x3E, 0x21)	(0xFFFF, 0x5555) → (0xAA7D, 0x270A)
<i>Key recovery</i>		
Secret / target key	[0x01, 0x02, 0x03, 0x04]	[0x1001, 0x2002, 0x3003, 0x4004]
Recovered key	[0x01, 0x02, 0x03, 0x04]	[0x1001, 0x2002, 0x3003, 0x4004]
Verification	✓ 3/3 pairs	✓ 3/3 pairs
<i>Timing</i>		
Offline (DS ^d build)	0.0006 s	—
Online phase	1.4816 s	48.622 s
Total time	1.4827 s	48.622 s
<i>Random-key robustness trials (seed = 42)</i>		
Trial 1	[0x39, 0x0C, 0x8C, 0x7D] PASS	$t=82.50$ s
Trial 2	[0x0F, 0x2F, 0x6F, 0x77] PASS	$t=21.85$ s
Trial 3	[0x03, 0x51, 0xD8, 0xAE] PASS	$t=4.78$ s
Pass rate	3 / 3	N/A
Outcome	✓ Success	✓ Success

Notes: DS^d = dec-side z -indexed lookup table; anchor bytes: $s_{d,0}=0xED$, $s_{d,1}=0x73$ (Simon-16/32 only). The 28-bit filter reduces the Simon-32/64 search space from 2^{64} to 2^{36} before matching. Robustness trials: Simon-16/32 only; timing variance reflects key position in GPU thread-partition ordering. Timings are wall-clock seconds, RTX 5050 (CC 12.0).

7. Conclusion

This paper presented a comprehensive GPU-accelerated cryptanalytic study of reduced-round Simon32/64, implementing brute-force key search, classical Meet-in-the-Middle, and Advance MitM (Correlated Sequence) attacks in CUDA. Key findings include:

1. **Finding 1:** GPU acceleration yields up to a 207x speedup over a single-threaded CPU for a 40-bit reduced search space. However, despite being highly parallelizable and requiring only $O(1)$ memory, a full exhaustive key search remains practically infeasible due to its $O(2^{64})$ time complexity.
2. **Finding 2:** The Meet-in-the-Middle (MitM) attack reduces the theoretical time complexity from $O(2^{64})$ to $O(2^{33})$ with an ideal space requirement of $O(2^{32})$. Applied to 4-round Simon32/64 with 8 key bits fixed, the optimal chunked GPU implementation projects a full key sweep in approximately 11 minutes (using an optimal FT chunk size of 2^{16} entries), demonstrating massive cryptographic time savings at the cost of severe, memory-bound host and CPU bottlenecks.
3. **Finding 3:** The MitM attack on SIMON 32/64, augmented by correlated-sequence amortisation and z -linear segment structure, demonstrates that the cipher’s algebraic regularity—specifically the rotational equivariance of its round function and the key-schedule linearity—can be exploited to reduce the effective attack complexity well below the naïve exhaustive bound.

This work presented a progressive complexity analysis of attacks against SIMON 32/64, spanning four tiers of increasing sophistication. A base-level software implementation established functional correctness and provided empirical cycle counts as a reference baseline. GPU-accelerated brute-force analysis demonstrated that raw parallelism, while effective at compressing wall-clock time across the full 2^{64} key space, offers no reduction in the *algorithmic* complexity and remains computationally intractable at the full parameter set. Classical meet-in-the-middle analysis on GPU confirmed the theoretical 2^{32} time–memory trade-off, validating that partitioning the key schedule at the $(k_0, k_1) \mid (k_2, k_3)$ boundary yields a measurable complexity gain over exhaustive search, even before structural optimisations are applied. Finally, the correlated-sequence MitM attack—exploiting the rotational equivariance $f(x \lll i) = f(x) \lll i$ of the SIMON round function and the z -linear segment structure to amortise $N^2(N^2 - 1)$ redundant round-function evaluations—achieved the sharpest complexity reduction of all tiers; its correctness and scaling behaviour were rigorously confirmed on the SIMON 8/16 toy model, where the compressed 2^{16} key space rendered the full attack pipeline observable and verifiable end-to-end. Taken together, the four tiers establish that the security margin of SIMON 32/64 is more sensitive to *structural* exploitation of its ARX symmetry than to raw computational scaling, underscoring the importance of rotational-coset analysis as a first-class evaluation criterion for lightweight cipher design.

Future work includes extension to Simon48/96 and Simon64/128, implementation of differential-linear combined attacks, and investigation of FPGA-based acceleration for memory-bandwidth-limited phases of the MitM attack.

References

-
- [1] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers. The SIMON and SPECK Families of Lightweight Block Ciphers. *IACR ePrint 2013/404*, 2013.
 - [2] W. Diffie and M. E. Hellman. Special feature: Exhaustive cryptanalysis of the NBS data encryption standard. *IEEE Computer*, 10(6):74–84, 1977.
 - [3] NVIDIA Corporation. CUDA Python Documentation, v12.2, 2023. <https://nvidia.github.io/cuda-python/latest/>.
 - [4] NSA Cybersecurity. SIMON and SPECK: Implementation guide. *National Security Agency Technical Report*, 2018.

- [5] R. Rohit, G. Gong, “Meet-in-the-Middle attack using Correlated Sequences and its Applications to Simon-like Ciphers[†],” *Department of Electrical and Computer Engineering, University of Waterloo*, .
- [6] R. Rohit, G. Gong, “Correlated Sequence Attack on Reduced-Round Simon-32/64 and Simeck-32/64,” *Department of Electrical and Computer Engineering, University of Waterloo*, .